MLA0101-AIES

LAB EXPERIMENTS

1. Bread first search (BFS)

A robot must find the shortest route to visit all rooms in a building level by level.

Code:

AIES Lab.py - C:/Users/chakr/AppData/Local/Programs/Python/Python312/AIES Lab.py...

File Edit Format Run Options Window Help

```
from collections import deque

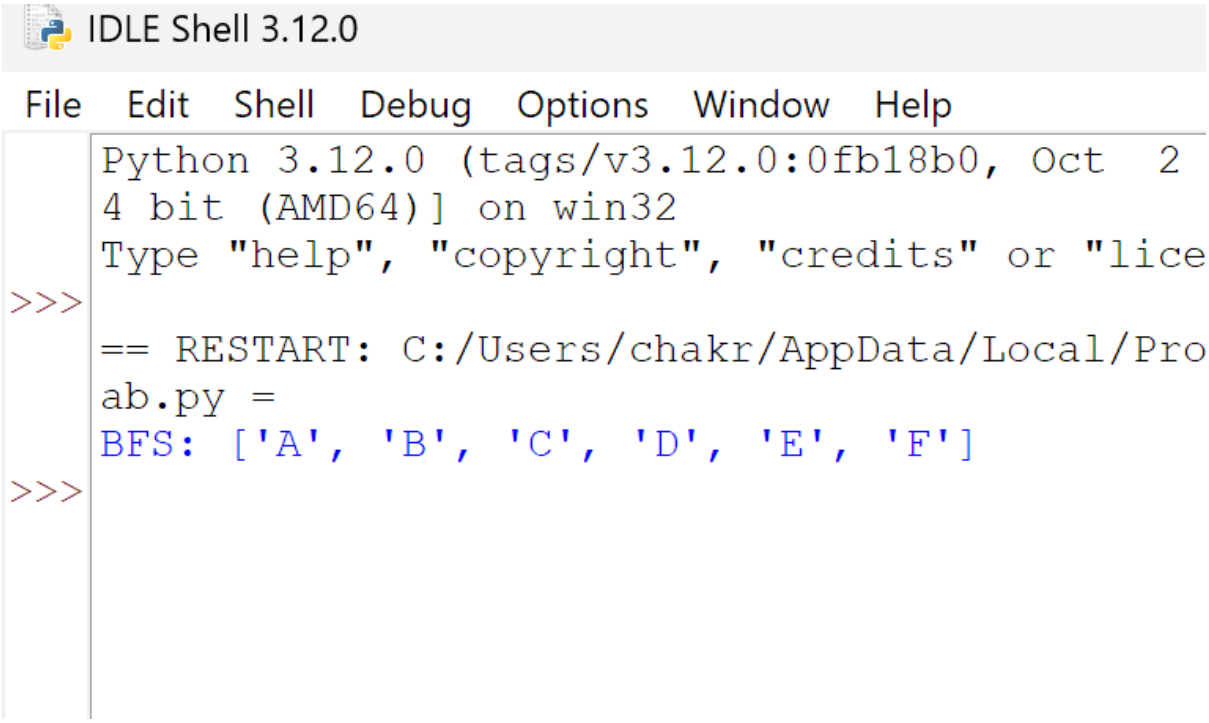
g = {'A':['B','C'], 'B':['D','E'],
     'C':['F'], 'D':[], 'E':[], 'F':[]}

q = deque(['A'])
visited = []

while q:
    n = q.popleft()
    if n not in visited:
        visited.append(n)
        q.extend(g[n])

print("BFS:", visited)
```

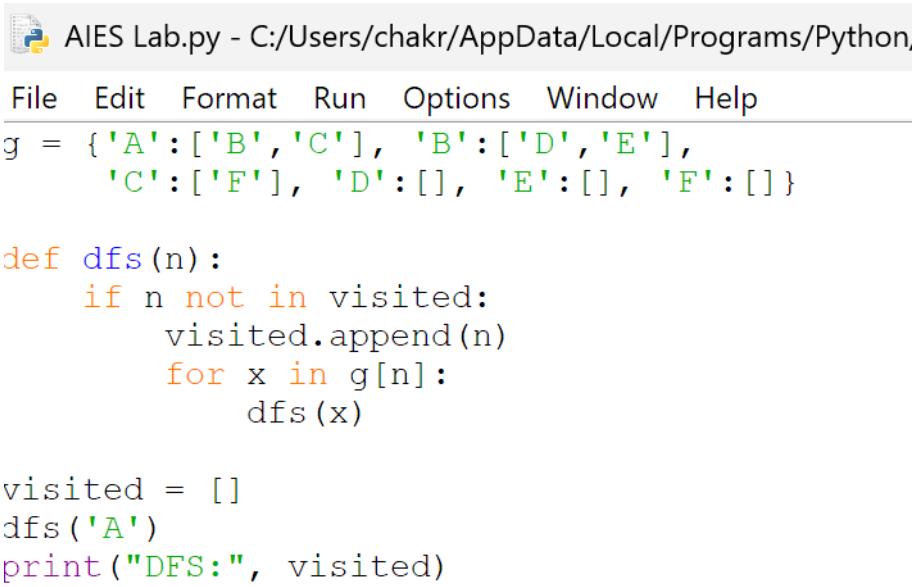
Output:



2. Depth First Search (DFS)

A rescue drone explores a cave by moving as deep as possible before backtracking.

Code:



Output:

```
>>>
= RESTART: C:/Users/chakr/AppData/Local
b.py
DFS: ['A', 'B', 'D', 'E', 'C', 'F']
>>>
```

3. Uniform Cost Search (UCS)

A logistics company wants to deliver goods with the minimum transportation cost.

Code:

AIES Lab.py - C:/Users/chakr/AppData/Local/Programs/Python/

File Edit Format Run Options Window Help

```
import heapq

g = {'A':[('B',1),('C',4)],
     'B':[('D',2)], 'C':[('D',1)], 'D':[]}

q = [(0, 'A')]
cost = {}

while q:
    c, n = heapq.heappop(q)
    if n in cost:
        continue
    cost[n] = c
    for x, w in g[n]:
        if x not in cost:
            heapq.heappush(q, (c+w, x))

print("Minimum cost to D:", cost.get('D'))
```

Output:

```
>>>
    = RESTART: C:/Users/chakr,
    b.py
    Minimum cost to D: 3
>>>
```

4. Best First Search

A GPS navigation system selects the next city based on the estimated distance to the destination.

Code:

AIES Lab.py - C:/Users/chakr/AppData/Local/Programs/Python/Python3

File Edit Format Run Options Window Help

```
import heapq

g = {'A':['B','C'], 'B':['D'], 'C':['D'], 'D':[]}
h = {'A':3, 'B':2, 'C':1, 'D':0}

q = [(h['A'], 'A')]
visited = []

while q:
    _, n = heapq.heappop(q)
    if n in visited:
        continue
    visited.append(n)
    if n == 'D':
        break
    for x in g[n]:
        heapq.heappush(q, (h[x], x))

print("Path:", visited)
```

Output:

```
>>>
= RESTART: C:/Users/chakr,
b.py
Path: ['A', 'C', 'D']
>>>
```

5. Greedy Best First Search

A warehouse robot chooses the nearest storage rack at every step to minimize travel time.

Code:

AIES Lab.py - C:/Users/chakr/AppData/Local/Programs/Python/Pythor

File Edit Format Run Options Window Help

```
import heapq

g = {'A':['B','C'], 'B':['D'], 'C':['D'], 'D':[]}
h = {'A':3, 'B':2, 'C':1, 'D':0}

q = [(h['A'], 'A')]
visited = []

while q:
    _, n = heapq.heappop(q)
    if n in visited:
        continue
    visited.append(n)
    if n == 'D':
        break
    for x in g[n]:
        if x not in visited:
            heapq.heappush(q, (h[x], x))

print("Visited:", visited)
```

Output:

```
>>>
    = RESTART: C:/Users/chakr/Aj
    b.py
    Visited: ['A', 'C', 'D']
>>>
```

6. A* Search Algorithm

A delivery robot must reach its destination using both actual travel cost and estimated remaining distance.

Code:

AIES Lab.py - C:/Users/chakr/AppData/Local/Programs/Python/Python31

File Edit Format Run Options Window Help

```
import heapq

g = {'A':[('B',1),('C',4)],
     'B':[('D',2)], 'C':[('D',1)], 'D':[]}
h = {'A':3, 'B':2, 'C':1, 'D':0}

q = [(h['A'], 0, 'A')]
cost = {}

while q:
    f, c, n = heapq.heappop(q)
    if n in cost:
        continue
    cost[n] = c
    for x, w in g[n]:
        if x not in cost:
            heapq.heappush(q, (c+w+h[x], c+w, x))

print("Minimum cost to D:", cost.get('D'))
```

Output:

```
>>> 
= RESTART: C:/Users/cha
b.py
Minimum cost to D: 3
>>> 
```

7. Minimax Algorithm

A chess-playing AI must determine the best move against an opponent.

Code:

AIES Lab.py - C:/Users/chakr/AppData/Local/Programs/Python/Python312/AIES Lab.py...

File Edit Format Run Options Window Help

```
def minimax(tree, depth, maximizing):
    if isinstance(tree, int):
        return tree
    values = [minimax(x, depth+1, not maximizing) for x in tree]
    return max(values) if maximizing else min(values)

tree = [[3, 5], [2, 9]]
print("Best value:", minimax(tree, 0, True))
```

Output:

```
>>> 
= RESTART: C:/Users/cha
b.py
Best value: 3
```

8. Alpha-Beta Pruning

A game AI needs to make faster decisions by avoiding unnecessary game-tree exploration.

Code:

AIES Lab.py - C:/Users/chakr/AppData/Local/Programs/Python/Python312/AIES Lab.py...

File Edit Format Run Options Window Help

```
def alphabeta(tree, alpha, beta, maxplayer):
    if isinstance(tree, int):
        return tree
    if maxplayer:
        v = -999
        for x in tree:
            v = max(v, alphabeta(x, alpha, beta, False))
            alpha = max(alpha, v)
            if alpha >= beta:
                break
        return v
    else:
        v = 999
        for x in tree:
            v = min(v, alphabeta(x, alpha, beta, True))
            beta = min(beta, v)
            if alpha >= beta:
                break
        return v

print("Best value:", alphabeta([[3,5],[2,9]], -999, 999, True)
```

Output:

```
>>>
= RESTART: C:/User
b.py
Best value: 3
```

9. Water Jug Problem

A person has two containers of different capacities and must measure an exact quantity of water.

Code:

AIES Lab.py - C:/Users/chakr/AppData/Local/Programs/Python/Pytho

File Edit Format Run Options Window Help

```
from collections import deque

q = deque([(0, 0)])
seen = set()

while q:
    a, b = q.popleft()
    if (a, b) in seen:
        continue
    seen.add((a, b))
    print(a, b)

    if a == 8 or b == 8:
        break

    q.extend([(11, b), (a, 9), (0, b), (a, 0),
              (max(0, a-(9-b)), min(9, a+b)),
              (min(11, a+b), max(0, b-(11-a)))])
```

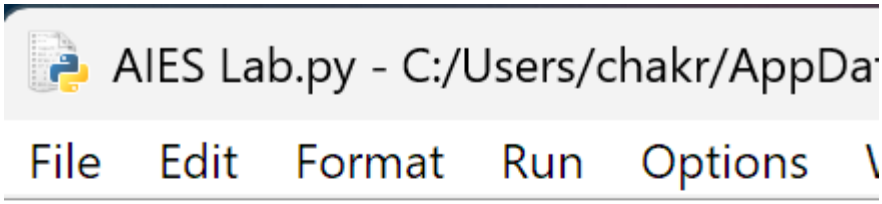
Output:

```
>>>
= RESTART: C:/Us
b.py
0 0
11 0
0 9
11 9
2 9
9 0
2 0
9 9
0 2
11 7
11 2
0 7
4 9
7 0
4 0
7 9
0 4
11 5
11 4
0 5
6 9
5 0
6 0
5 9
0 6
11 3
11 6
0 3
8 9
```

10.Towers of Hanoi

A robotic arm transfers stacked containers while following movement constraints.

Code:




```
def hanoi(n, a, b, c):
    if n == 1:
        print(a, "->", c)
        return
    hanoi(n-1, a, c, b)
    print(a, "->", c)
    hanoi(n-1, b, a, c)

hanoi(3, 'A', 'B', 'C')
```

Output:

```
>>>
    = RESTART: C:
    b.py
    A -> C
    A -> B
    C -> B
    A -> C
    B -> A
    B -> C
    A -> C
```